



IFPB - Campus Campina Grande

CURSO: Engenharia de Computação

Bandline: Sistema de Visualização de Álbuns

**Trabalho da disciplina Testes de Software,
ministrada pelo Professor Breno Romero.
Semestre: 2025.2**

Equipe:

Miguel Ryan Dantas de Freitas

Felipe Luiz de Lima

Carlos Elias Fialho de Lima

Marcus Cauê de Farias Barbosa

Kátia Emmanuelle Gomes da Silva

Campina Grande, 11 de dezembro de 2025

Sumário:

- 1 – Definições do Sistema (Funcionalidades do sistema, sua aplicação e arquiteturas)
- 2 – Análise de Requisitos
 - 2.1 – Requisitos Funcionais
 - 2.3 – Requisitos Não Funcionais
 - 2.4 – Requisitos Legais/Compulsórios (Leis, Normativas, Regimentos)
- 3 – Seleção de funcionalidades para testes (mínimo 2)
- 4 – Justificativa das escolhas
- 5 – Cenários de testes e Casos de Testes
- 6 – Relação dos testes executados (Testes que passaram e Testes que falharam com as indicações de correção)
- 7 – Telas dos testes executados
- 8 – Código fonte / link GitHub

1. DEFINIÇÕES DO SISTEMA (FUNCIONALIDADES DO SISTEMA, SUA APLICAÇÃO E ARQUITETURAS)

1.1 Aplicação e Arquiteturas

O projeto é concebido para oferecer uma experiência de busca e consumo de conteúdo musical focada em artistas, álbuns e faixas. Seu propósito central é permitir que os usuários busquem artistas utilizando o catálogo do Spotify, enriqueçam esses detalhes com informações descritivas do Last.fm, navegue por álbuns e faixas, e gerenciar coleções pessoais como favoritos, histórico de buscas e álbuns concluídos, além de manter um perfil de usuário.

As principais funcionalidades do sistema abrangem todo o ciclo de vida do usuário e consumo de dados. O módulo de autenticação inclui cadastro, login, logout e verificação de sessão, com suporte tanto para autenticação local quanto via OAuth do Google. No que tange ao catálogo, há a funcionalidade de busca de artistas, a consulta de detalhes de artistas (que inclui descrição traduzida e artistas relacionados), a listagem de álbuns de um artista ordenados por data e a consulta das faixas de um álbum específico. Para a gestão pessoal, o sistema oferece histórico de buscas de artistas (limitado a cinquenta registros), funcionalidades completas de gestão de álbuns favoritos (criação, listagem, verificação e remoção) e um módulo similar para o rastreamento de álbuns concluídos. O usuário também pode consultar e atualizar seu perfil (username, email, avatar e bio). Por fim, a documentação interativa da API, construída com OpenAPI e visualizada com Scalar, é servida em uma rota do backend.

A arquitetura lógica é dividida em frontend, backend e persistência. O Frontend é uma Single Page Application (SPA) desenvolvida em Svelte, que consome a API HTTP do backend. O roteamento é feito de forma manual e o aplicativo possui componentes dedicados para favoritos, histórico, playlists e perfil. O Backend é uma API REST construída com Express, suportando estratégias local e Google OAuth para autenticação. A Persistência utiliza PostgreSQL como banco de dados relacional, orquestrado pelo Prisma ORM.

O fluxo de dados segue o padrão de aplicações web modernas: o cliente SPA chama os *endpoints* REST, autenticando-se via cookies de sessão. Os dados de

domínio (como histórico e favoritos) são persistidos no PostgreSQL através do Prisma. Já os dados de catálogo (artistas, álbuns, faixas) são lidos sob demanda diretamente das APIs externas Spotify/Last.fm, e então filtrados e mapeados antes de serem retornados ao cliente.

A qualidade é assegurada por testes construídos com Vitest, que apontam o Prisma Client para um banco de dados de teste, e utilizam *mocks* de API para simular as integrações externas.

2. ANÁLISE DE REQUISITOS

2.1 Requisitos Funcionais

- RF01 — Permitir cadastro de usuário com username, email e senha; rejeitar duplicados e campos faltantes.
- RF02 — Realizar login e manter sessão via cookie; realizar logout.
- RF03 — Oferecer login via OAuth Google (redirecionamento e callback).
- RF04 — Buscar artistas no Spotify a partir de query, retornando lista básica limitada a 10 itens.
- RF05 — Exibir detalhes de artista com descrição localizada (en/pt/es) e artistas relacionados do Last.fm.
- RF06 — Listar álbuns de um artista ordenados por data de lançamento.
- RF07 — Listar faixas de um álbum com metadados essenciais.
- RF08 — Registrar histórico de buscas de artistas e listar últimos 50 itens do usuário.
- RF09 — Gerenciar favoritos de álbuns: criar, listar (máx. 50), verificar vários IDs e remover.
- RF10 — Gerenciar álbuns concluídos: criar, listar (máx. 50), verificar vários IDs e remover.
- RF11 — Permitir consulta e atualização de perfil (username, email, bio, avatar).
- RF12 — Disponibilizar documentação da API.

2.2 Requisitos Não Funcionais

- RNF01 — Persistência em banco de dados PostgreSQL (dados de catálogo não são persistidos).
- RNF02 — Senhas precisam ser criptografadas no banco de dados
- RNF03 — Autenticação baseada em sessão.
- RNF04 — Limites defensivos: buscas de artista exigem query ≥ 2 caracteres; listas de histórico/favoritos/completos são limitadas a 50 itens; busca de artistas limitada a 10 resultados.
- RNF05 — Tratamento de falhas: respostas 4xx para validações e 5xx para erros internos; logging simples em console para diagnósticos.
- RNF06 — Testabilidade: suíte de testes usando banco de teste isolado e mocks das integrações Spotify/Last.fm.
- RNF07 — Documentação servida pelo próprio backend; interface visual da mesma.
- RNF08 — Configuração de variáveis de ambiente para evitar exposição do projeto.

2.3 Requisitos Legais/Compulsórios

- RL01 — Conformidade com LGPD: coletar apenas dados necessários (username, email, avatar opcional, bio, histórico/favoritos do próprio usuário); oferecer meios para atualização e exclusão via endpoints de perfil e exclusões específicas.
- RL02 — Senhas devem ser armazenadas exclusivamente como hash e nunca em texto plano; sessão mantida por cookie não exposto a scripts.
- RL03 — Uso de APIs de terceiros deve seguir os Termos de Uso do Spotify Web API e da Last.fm (inclui limites de rate, proibição de redistribuição não autorizada e requisitos de atribuição).
- RL04 — Login OAuth Google deve obedecer políticas da Google (escopos mínimos, tratamento de erro de callback).
- RL05 — Cookies de sessão exigem aviso de uso ao usuário final e aderência às políticas de privacidade aplicáveis.

3. SELEÇÃO DE FUNCIONALIDADES PARA TESTES

3.1.1 Rota de Favoritos da Aplicação

3.1.2 Rota de Busca por Artistas da Aplicação

4. JUSTIFICATIVA DAS ESCOLHAS

4.1.1 Justificativa da Escolha da Rota de Favoritos da Aplicação

Esta rota foi selecionada como a principal demonstração da robustez do backend proprietário porque é um endpoint servido e mantido pela minha própria API, ao contrário de dados hidratados por uma fonte externa. A escolha permite uma cobertura de testes do backend, essencial para provar a integridade da lógica de negócios central. Nela, foram aplicados Testes de Unidade, Testes de Integração com o banco de dados e, crucialmente, Testes de Segurança (Autenticação e Autorização) sobre o ciclo de vida CRUD (Create, Read, Update, Delete) do dado. Validando a capacidade da API em gerenciar estados dinâmicos do usuário de forma segura, consistente e bem testada, garantindo a qualidade do schema e a persistência dos dados críticos.

4.1.2 Justificativa da Escolha da Rota de Busca por Artistas da Aplicação

A rota de Busca por Artistas foi escolhida para demonstrar a capacidade de integração frente a serviços de terceiros. Por ser uma rota que depende de uma API externa (Spotify e LastFM). A análise dos testes desta rota no documento comprova a eficácia de mecanismos como tratamento de erros de rede, mitigação de latência através de estratégias de caching (validando a redução de Rate Limit e gargalos de requisição), e a normalização eficiente do payload da API externa para o schema de dados interno. Validando que a aplicação é capaz de interagir com fontes de dados massivas e variáveis de forma estável e segura, sem comprometer o desempenho da experiência do usuário ou expor credenciais (data leaking) para o cliente.

5. CENÁRIOS DE TESTES E CASOS DE TESTES

5.1 Cenários e Casos de Teste da Rota de Favoritos

Esta seção detalha os testes realizados na rota de Favoritos, que é um endpoint Self-Hosted essencial para a lógica de estado do usuário. Os cenários de teste demonstram a segurança, a validação de dados e a integridade das operações de persistência (CRUD), garantindo que os dados críticos do usuário sejam manipulados de forma segura e consistente. O foco principal reside na validação da Autorização (HTTP 401), onde todas as operações (adicionar, listar, remover, verificar) devem ser estritamente bloqueadas para usuários não autenticados. Adicionalmente, são testadas as operações de sucesso (HTTP 200) e os cenários de exceção e conflito (HTTP 400, 409, 404), que garantem o tratamento adequado de *schemas* inválidos, tentativas de duplicação e remoção de itens inexistentes.

5.1.1 Cenário de Teste 01: Adição de Favoritos (POST /favorites)

Este cenário engloba todos os testes relacionados à criação de um novo registro de favorito, cobrindo segurança, sucesso na inserção, validação de campos obrigatórios e prevenção de duplicidade.

5.1.1.1 Caso de Teste 01.01: Validação de Segurança na Adição

ID do Teste	CT-FAV-POST-01
Descrição da Ação	Tentar realizar uma requisição para adicionar um favorito sem que o usuário esteja autenticado no sistema.
Pré-Condição	O usuário não possui sessão ativa (sem token/cookie).
Entrada	Rota: POST /favorites Headers: Sem Cookies Body: { albumId: "album1", albumName: "Album 1" }

Saída Esperada	Status: 401 Unauthorized Body: { error: "Não Autenticado" }
-----------------------	--

5.1.1.2 Caso de Teste 01.02: Adição de Favorito com Sucesso

ID do Teste	CT-FAV-POST-02
Descrição da Ação	Realizar login com um usuário válido e adicionar um álbum completo aos favoritos.
Pré-Condição	Usuário criado e logado (Cookie de sessão obtido).
Entrada	Rota: POST /favorites Headers: Cookie: [session_cookie] Body: { albumId: "1To7...", albumName: "OK Computer", albumImage: "...", albumTracks: [...] }
Saída Esperada	Status: 200 OK Body: Objeto favorite contendo os dados enviados e userId correto. Mensagem: "Álbum favoritado com sucesso"

5.1.1.3 Caso de Teste 01.03: Validação de Campos Obrigatórios

ID do Teste	CT-FAV-POST-03
Descrição da Ação	Tentar adicionar um favorito omitindo um campo obrigatório (neste caso, albumName).
Pré-Condição	Usuário criado e logado.

Entrada	Rota: POST /favorites Headers: Cookie: [session_cookie] Body: { albumId: "album1" } // albumName faltando
Saída Esperada	Status: 400 Bad Request Body: { error: "albumId e albumName são obrigatórios" }

5.1.1.4 Caso de Teste 01.04: Prevenção de Duplicidade (Conflito)

ID do Teste	CT-FAV-POST-04
Descrição da Ação	Tentar adicionar um álbum que já foi previamente salvo nos favoritos do usuário.
Pré-Condição	Usuário logado e registro de favorito já existente no banco de dados para o ID "existing_album".
Entrada	Rota: POST /favorites Headers: Cookie: [session_cookie] Body: { albumId: "existing_album", albumName: "Existing Album" }
Saída Esperada	Status: 409 Conflict Body: { error: "Álbum já está nos favoritos" } O objeto favorite existente deve ser retornado.

5.1.2 Cenário de Teste 02: Listagem de Favoritos (GET /favorites)

Este cenário cobre a recuperação dos dados salvos no banco de dados, verificando a proteção da rota e o formato da resposta para listas vazias e preenchidas.

5.1.2.1 Caso de Teste 02.01: Validação de Segurança na Listagem

ID do Teste	CT-FAV-GET-01
Descrição da Ação	Tentar listar os favoritos sem estar autenticado.
Pré-Condição	O usuário não possui sessão ativa.
Entrada	Rota: GET /favorites Headers: Sem Cookies
Saída Esperada	Status: 401 Unauthorized Body: { error: "Não Autenticado" }

5.1.2.2 Caso de Teste 02.02: Retorno de Lista Vazia

ID do Teste	CT-FAV-GET-02
Descrição da Ação	Solicitar a lista de favoritos de um usuário novo que ainda não salvou nenhum álbum.
Pré-Condição	Usuário criado e logado, sem registros na tabela Favorites.
Entrada	Rota: GET /favorites Headers: Cookie: [session_cookie]
Saída Esperada	Status: 200 OK Body: { favorites: [] } (Array vazio).

5.1.2.3 Caso de Teste 02.03: Retorno de Lista Preenchida e Ordenada

ID do Teste	CT-FAV-GET-03
--------------------	---------------

Descrição da Ação	Solicitar a lista de favoritos de um usuário que possui múltiplos registros.
Pré-Condição	Usuário criado, logado e com 2 álbuns inseridos no banco de dados.
Entrada	Rota: GET /favorites Headers: Cookie: [session_cookie]
Saída Esperada	Status: 200 OK Body: Array favorites contendo 2 objetos. Validação: O userId dos itens deve corresponder ao usuário logado.

5.1.3 Cenário de Teste 03: Remoção de Favoritos (DELETE /favorites)

Este cenário valida o fluxo de exclusão, garantindo que apenas usuários autenticados possam remover itens, que o ID seja fornecido e que tentativas de remover itens inexistentes sejam tratadas.

5.1.3.1 Caso de Teste 03.01: Validação de Segurança na Remoção

ID do Teste	CT-FAV-DEL-01
Descrição da Ação	Tentar remover um favorito sem estar autenticado.
Pré-Condição	O usuário não possui sessão ativa.
Entrada	Rota: DELETE /favorites Body: { albumId: "album1" }

Saída Esperada	Status: 401 Unauthorized Body: { error: "Não Autenticado" }
-----------------------	--

5.1.3.2 Caso de Teste 03.02: Remoção com Sucesso

ID do Teste	CT-FAV-DEL-02
Descrição da Ação	Remover um favorito existente fornecendo o ID correto.
Pré-Condição	Usuário logado e álbum "album_to_delete" previamente salvo no banco.
Entrada	Rota: DELETE /favorites Headers: Cookie: [session_cookie] Body: { albumId: "album_to_delete" }
Saída Esperada	Status: 200 OK Body: { message: "Favorito removido com sucesso", deleted: { count: 1 } }

5.1.3.3 Caso de Teste 03.03: Validação de Parâmetro Obrigatório

ID do Teste	CT-FAV-DEL-03
Descrição da Ação	Tentar enviar uma requisição de remoção sem especificar qual álbum deve ser removido.
Pré-Condição	Usuário logado.

Entrada	Rota: DELETE /favorites Headers: Cookie: [session_cookie] Body: {} (Objeto vazio)
Saída Esperada	Status: 400 Bad Request Body: { error: "albumId é obrigatório" }

5.1.3.4 Caso de Teste 03.04: Tratamento de Item Inexistente

ID do Teste	CT-FAV-DEL-04
Descrição da Ação	Tentar remover um álbum que não consta na lista de favoritos do usuário.
Pré-Condição	Usuário logado e álbum "nonexistent_album" NÃO existe no banco.
Entrada	Rota: DELETE /favorites Headers: Cookie: [session_cookie] Body: { albumId: "nonexistent_album" }
Saída Esperada	Status: 404 Not Found Body: { error: "Favorito não encontrado" }

5.1.4 Cenário de Teste 04: Verificação de Status em Lote (POST /favorites/check)

Este cenário testa a funcionalidade auxiliar que permite ao front-end verificar, em uma única requisição, quais álbuns de uma lista já foram favoritados pelo usuário.

5.1.4.1 Caso de Teste 04.01: Validação de Segurança na Verificação

ID do Teste	CT-FAV-CHECK-01
Descrição da Ação	Tentar verificar o status de favoritos sem estar autenticado.
Pré-Condição	O usuário não possui sessão ativa.
Entrada	Rota: POST /favorites/check Body: { albumIds: ["album1"] }
Saída Esperada	Status: 401 Unauthorized Body: { error: "Não Autenticado" }

5.1.4.2 Caso de Teste 04.02: Verificação de Lista de IDs

ID do Teste	CT-FAV-CHECK-02
Descrição da Ação	Enviar uma lista contendo IDs de álbuns favoritados e não favoritados para verificar quais retornam.
Pré-Condição	Usuário logado, possuindo "album1" e "album2" nos favoritos, mas NÃO possuindo "album3".
Entrada	Rota: POST /favorites/check Headers: Cookie: [session_cookie] Body: { albumIds: ["album1", "album2", "album3"] }

Saída Esperada	Status: 200 OK Body: { favoritelds: [...] } Validação: O array retornado deve conter "album1" e "album2", mas NÃO deve conter "album3".
-----------------------	--

5.1.4.3 Caso de Teste 04.03: Validação de Tipo de Dado (Array)

ID do Teste	CT-FAV-CHECK-03
Descrição da Ação	Enviar a requisição com o campo albumIds em um formato inválido (não sendo um array).
Pré-Condição	Usuário logado.
Entrada	Rota: POST /favorites/check Headers: Cookie: [session_cookie] Body: { albumIds: "not_an_array" }
Saída Esperada	Status: 400 Bad Request Body: { error: "albumIds é obrigatório e deve ser um array" }

5.2 Cenários e Casos de Teste da Rota de Busca por Artistas

Esta seção detalha os testes realizados na rota de Busca por Artistas, que demonstra a resiliência da aplicação na integração com APIs Externas (Third-Party Endpoint). Os cenários de teste focam na otimização de requisições, no tratamento de erros da API de terceiros e na validação de entrada. É testada a Validação Mínima de Entrada, garantindo que consultas inválidas (vazias ou muito curtas) sejam interceptadas localmente para evitar chamadas desnecessárias à API externa, preservando o limite de Rate Limit. O cenário de Integração com Sucesso (HTTP 200) valida a comunicação, o consumo e a correta normalização dos dados

externos. Por fim, o cenário de Resiliência a Falhas Externas (HTTP 500) verifica se o backend consegue lidar e traduzir erros da API de terceiros para uma resposta genérica e segura ao cliente.

5.2.1 Cenário de Teste 05: Validação de Entrada (Otimização de Requisições)

Este cenário foca na validação dos parâmetros de entrada antes de acionar serviços externos. O objetivo é garantir que a aplicação não desperdice recursos ou limites de requisição (*rate limits*) da API do Spotify com consultas vazias ou insuficientes.

5.2.1.1 Caso de Teste 01.01: Busca sem Parâmetro de Query

ID do Teste	CT-BUSCA-VAL-01
Descrição da Ação	Tentar realizar uma busca sem fornecer o parâmetro obrigatório de texto (q).
Pré-Condição	O servidor está rodando e a rota está acessível.
Entrada	Rota: GET /searchArtists Query Params: Nenhum (vazio)
Saída Esperada	Status: 200 OK Body: { artists: [] } Comportamento: A função fetch (mock) NÃO deve ser chamada.

5.2.1.2 Caso de Teste 01.02: Busca com Query Muito Curta

ID do Teste	CT-BUSCA-VAL-02
Descrição da Ação	Tentar realizar uma busca fornecendo um termo com tamanho insuficiente (ex: apenas 1 caractere), que não justifica uma chamada à API externa.
Pré-Condição	O servidor está rodando.
Entrada	Rota: GET /searchArtists Query Params: q=a
Saída Esperada	Status: 200 OK Body: { artists: [] } Comportamento: A função fetch (mock) NÃO deve ser chamada.

5.2.2 Cenário de Teste 06: Integração com API Externa (Sucesso e Erro)

Este cenário valida o núcleo da funcionalidade: a comunicação com a API do Spotify via mock, o tratamento dos dados recebidos e a resiliência da aplicação frente a falhas no serviço de terceiro.

5.2.2.1 Caso de Teste 02.01: Busca de Artistas com Sucesso

ID do Teste	CT-BUSCA-INT-01
Descrição da Ação	Realizar uma busca válida que simula uma resposta bem-sucedida da API do Spotify.

Pré-Condição	O Mock do fetch está configurado para retornar ok: true, status 200 e um JSON válido de artistas (mockSpotifyArtistsSearchResponse).
Entrada	Rota: GET /searchArtists Query Params: q=Radiohead
Saída Esperada	Status: 200 OK Body: Objeto contendo a lista de artistas formatada/normalizada. Comportamento: A função fetch deve ter sido chamada com o token e a query corretos.

5.2.2.2 Caso de Teste 06.02: Tratamento de Erro na API Externa (HTTP 500)

ID do Teste	CT-BUSCA-INT-02
Descrição da Ação	Realizar uma busca válida, mas simulando uma falha interna nos servidores do Spotify (Erro 500).
Pré-Condição	O Mock do fetch está configurado para retornar ok: false, status 500 e um JSON de erro { error: "Internal Server Error" }.
Entrada	Rota: GET /searchArtists Query Params: q=Radiohead

Saída Esperada	Status: 500 Internal Server Error Body: { error: "Erro ao buscar artistas" } Comportamento: A aplicação deve capturar o erro externo e retornar uma mensagem amigável ao cliente, sem "quebrar" (<i>crash</i>) o servidor.
-----------------------	---

6. RELAÇÃO DOS TESTES EXECUTADOS (TESTES QUE PASSARAM E TESTES QUE FALHARAM COM AS INDICAÇÕES DE CORREÇÃO)

A execução dos testes de integração utilizando o framework Vitest resultou em um sucesso completo (100% de cobertura nos casos de teste definidos) para as rotas críticas de Favoritos e Busca por Artistas. A tabela 6.1 a seguir consolida os resultados obtidos, demonstrando a estabilidade e a robustez atual da API Self-Hosted.

6.1 Relação dos Testes Executados (Tabela)

Arquivo de Teste (Rota Principal)	Número de Cenários/Suítes	Número de Casos de Teste (CTs)	Resultado
Rota de Favoritos da Aplicação	4 Suítes/Cenários (POST, GET, DELETE, /check)	14	100% Passou
Rota de Busca por Artistas da Aplicação	1 Suíte/Cenário (GET /searchArtists)	4	100% Passou

Total	5	18	Sucesso Total
--------------	---	----	------------------

6.2 Análise e Discussão da Evolução dos Testes

O resultado atual de 100% de sucesso em todos os casos de teste é um indicativo de evolução do projeto e qualidade da implementação da API. É crucial ressaltar que a efetividade desses testes não se limita à sua aprovação final, mas sim ao papel fundamental que desempenharam ao longo do ciclo de desenvolvimento do projeto.

Em versões anteriores e nas etapas iniciais de desenvolvimento das rotas, diversos testes falharam, expondo bugs, vulnerabilidades de segurança (como a falta do HTTP 401) e falhas na validação do schema de dados (Erros 400). Essas falhas iniciais, corretamente capturadas pelo Vitest, foram o motor para a evolução e correção da implementação.

Portanto, esse desenvolvimento atrelado à testes permitiu a refatoração e a melhoria contínua do código. Os testes executados não apenas validam a funcionalidade atual, mas também garantem a regressão, assegurando que as funcionalidades anteriormente implementadas (CRUD de Favoritos e Integração com Spotify) continuem a operar de forma segura e consistente, conforme os requisitos de negócio e técnicos.

7. TELAS DOS TESTES EXECUTADOS

Os testes foram desenvolvidos utilizando o framework Vitest. Para uma visualização em tempo real da execução e dos resultados, foi empregada a Interface Gráfica (UI) nativa do Vitest.

7.1 Suítes de Teste da Aplicação Completa

```
✓ routes/albums.test.ts (3) 809ms
✓ routes/artists.test.ts (3) 2103ms
✓ routes/auth.test.ts (10) 870ms
✓ routes/completedAlbums.test.ts (12) 1162ms
✓ routes/edge-cases.test.ts (5) 5210ms
✓ routes/external-apis-nodb.test.ts (2)
✓ routes/favorites.test.ts (14) 1433ms
✓ routes/history.test.ts (7) 742ms
✓ routes/profile.test.ts (7) 611ms
✓ routes/search.test.ts (4)
✓ routes/tracks.test.ts (3)

Test Files 11 passed
Tests 70 passed
Start at 13:40:37
Duration 16.65s (transform 167ms, setup 273ms, collect 2.02s, tests 13.07s, environment 1ms, prepare 469ms)
```

O Vitest UI oferece um painel interativo que exibe a hierarquia completa dos arquivos e casos de teste. O resultado da execução reflete a estrutura modular da aplicação, separada em suítes lógicas por arquivo:

1. routes/favorites.test.ts: Contém as suítes para a rota de favoritos da aplicação.
2. routes/search.test.ts: Contém as suítes para a funcionalidade de busca por artistas da aplicação.

O output do console demonstra que todos os testes definidos foram concluídos com sucesso (indicado pelo ✓ verde), totalizando 14 testes na rota de Favoritos e 4 testes na rota de Busca.

7.2 Suíte de Testes das Funcionalidades Escolhidas

7.2.1 Suíte de Testes da Rota de Favoritos da Aplicação

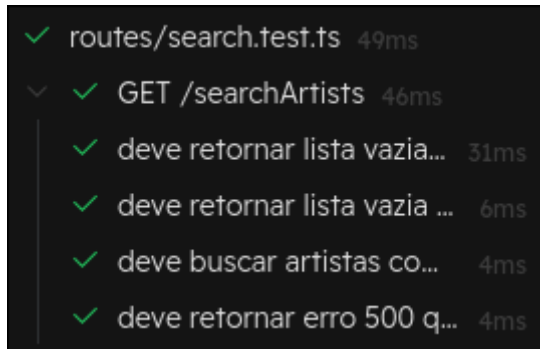


✓	routes/favorites.test.ts	1474ms
✓	POST /favorites	486ms
✓	deve retornar erro 401 ...	36ms
✓	deve adicionar favorito...	162ms
✓	deve retornar erro 400...	142ms
✓	deve retornar erro 409...	145ms
✓	GET /favorites	286ms
✓	deve retornar erro 401 q...	6ms
✓	deve retornar lista vazi...	140ms
✓	deve retornar favorito...	140ms
✓	DELETE /favorites	419ms
✓	deve retornar erro 401 q...	8ms
✓	deve remover favorito ...	138ms
✓	deve retornar erro 400...	137ms
✓	deve retornar erro 404...	137ms
✓	POST /favorites/check	282ms
✓	deve retornar erro 401 q...	6ms
✓	deve retornar lista de I...	138ms
✓	deve retornar erro 400...	137ms

A suíte de Favoritos (routes/favorites.test.ts) é subdividida em blocos de testes (describe) que cobrem as quatro operações principais, conforme visualizado na interface:

- POST /favorites: Testes de adição, segurança (HTTP 401), validação de schema (HTTP 400) e tratamento de conflito (HTTP 409).
- GET /favorites: Testes de listagem, segurança (HTTP 401) e formato de retorno (lista vazia ou preenchida).
- DELETE /favorites: Testes de remoção, segurança, validação de campo obrigatório e tratamento de item não encontrado (HTTP 404).
- POST /favorites/check: Testes da funcionalidade de verificação em lote de IDs.

7.2.2 Suíte de Testes da Rota de Busca por Artistas da Aplicação



A suíte de Busca (routes/search.test.ts) foca na integração e resiliência, contendo o bloco GET /searchArtists, que testa:

- **Otimização de Query:** Retorno de lista vazia para consultas curtas ou ausentes, evitando chamadas desnecessárias à API externa.
- **Integração de Sucesso:** Validação da comunicação correta com a API externa (Spotify Mock).
- **Tratamento de Erros:** Garantia de que falhas externas (simulação de HTTP 500) sejam interceptadas e traduzidas para uma resposta genérica segura.

7.3 Execução e Acesso à Interface Gráfica

O Vitest UI é a ferramenta recomendada para o acompanhamento interativo do desenvolvimento, permitindo que o desenvolvedor isole suítes, depure falhas e monitore a cobertura de código de forma intuitiva.

7.3.1 Procedimento Técnico de Execução

Para rodar os testes e visualizar esta interface de maneira técnica e robusta, o procedimento é o seguinte, no terminal, dentro da pasta 'tests' da aplicação:

```
npx vitest --ui
```

8. CÓDIGO FONTE / LINK GITHUB

<https://github.com/athavus/bandline>

8.1 Configuração do Projeto

É IMPORTANTE RESSALTAR QUE O PROJETO NECESSITA DE UM ARQUIVO .env DENTRO DA PASTA DE TESTES (para configuração segura do banco de dados, APIs e etc.), segue abaixo um documento fornecido para a correção, naturalmente, é importante não expôr esses dados, por serem minhas próprias credenciais:

8.1.1 tests/.env

```
SPOTIFY_CLIENT_ID=35a8c8ba6cc64537b064366f73d41e55
SPOTIFY_CLIENT_SECRET=d3e79b1de53a4e0392b2a3bca4efab00

GOOGLE_CLIENT_ID=868209435707-c74ergnnb5i2m1e0kijqjinj3oltgeid.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPIX-YcKCqtF5O8-hfjKSWM2fED3jfoNq

SESSION_SECRET=f12b3ab18cd2775ad01f3ce375665546ca13647c7673a76bdb2c08494589292a

GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

LASTFM_API_KEY=27b6a83dd8047cd5969e27ac26c3d1b9

DATABASE_URL="postgresql://test:test@localhost:5432/test"

PORT=3000
```

Se for necessário testar o resto da aplicação, o resto dos arquivos sensíveis e necessários é:

8.1.2 server/.env

```
SPOTIFY_CLIENT_ID=35a8c8ba6cc64537b064366f73d41e55
SPOTIFY_CLIENT_SECRET=d3e79b1de53a4e0392b2a3bca4efab00

GOOGLE_CLIENT_ID=868209435707-c74ergnnb5i2m1e0kijqjinj3oltgeid.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPIX-YcKCqtF5O8-hfjKSWM2fED3jfoNq
```


SESSION_SECRET=f12b3ab18cd2775ad01f3ce375665546ca13647c7673a76bdb2c
08494589292a

GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

LASTFM_API_KEY=27b6a83dd8047cd5969e27ac26c3d1b9

DATABASE_URL="postgresql://bandline_user:1234@localhost:5432/bandline_db"

PORT=3000

8.1.3 client/.env

VITE_API_URL=http://localhost:3000